

# **Intelligenza Artificiale**

## **“Solving problems by searching”**

(part 1)

Nicola Bellotto

A.A. 2025/2026

# Lecture outline

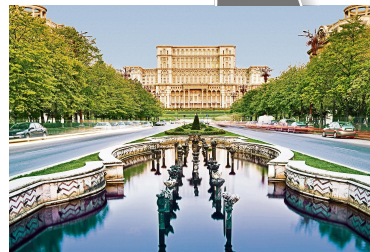
- Problem solving agents
- Problem examples
- Search algorithms
- Uninformed search

Slides partly based on Russell & Norvig instructors material <http://aima.cs.berkeley.edu/instructors.html>

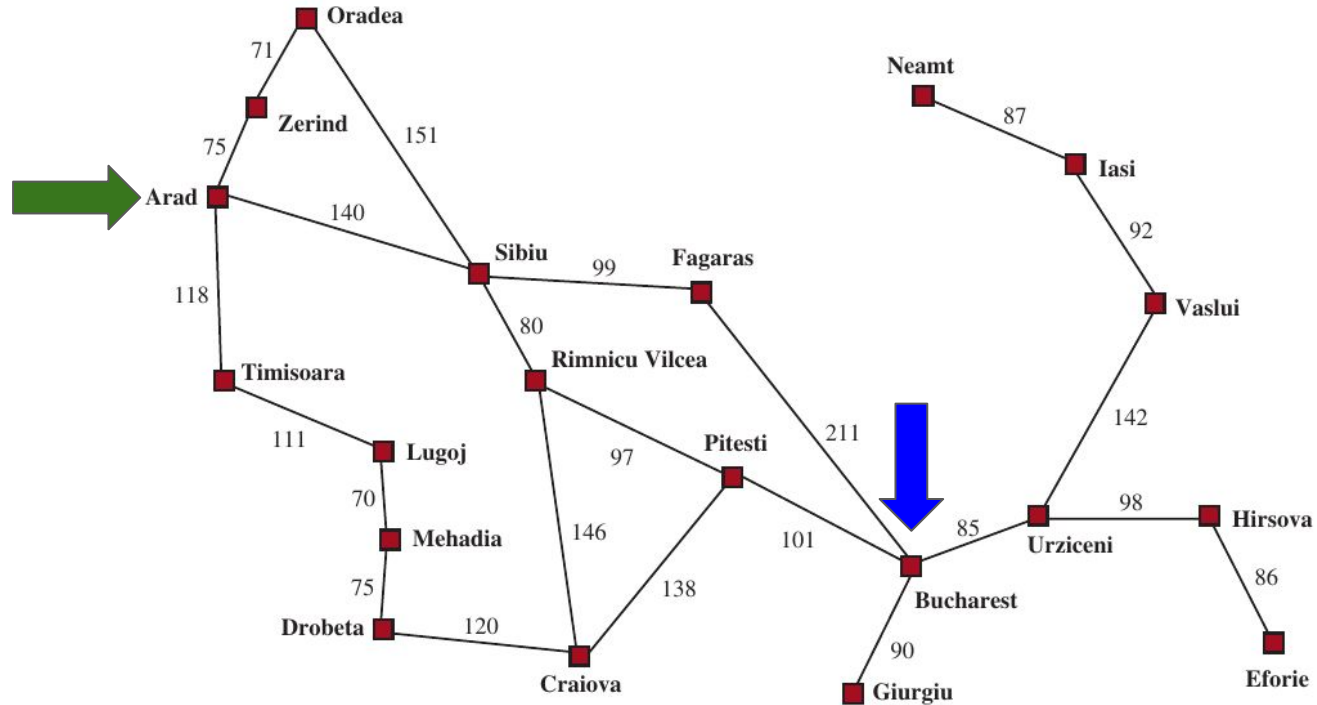
# Example: Romania

On holiday in Romania, currently in Arad.  
Flight leaves tomorrow from Bucharest.

- **Formulate goal:**
  - be in Bucharest
- **Formulate problem:**
  - states: various cities
  - actions: drive between cities
- **Search solution:**
  - sequence of cities
    - e.g. Arad → Sibiu → Fagaras → Bucharest
- **Execute** sequence of actions

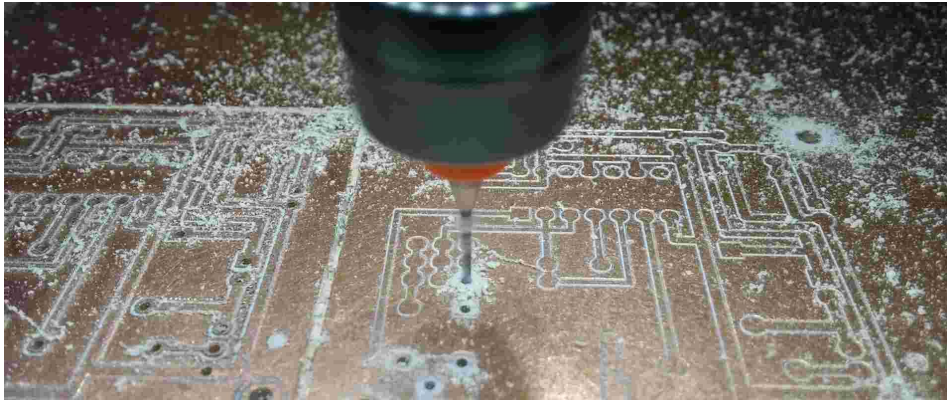


# Example: Romania



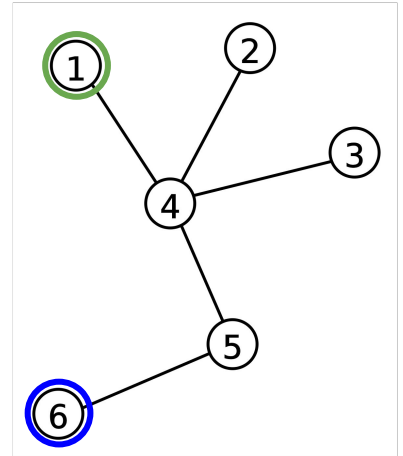
# Example: Travelling Salesperson Problem (TSP)

- Every vertex of a graph must be visited
  - could be a map like the previous one...
  - ... or could be an automatic drilling machine for printed circuit boards (PCBs)



# Problem solving agents

- **Special case of goal-based agents** that needs to plan ahead
  - Consider a sequence of actions leading to a goal
- Use atomic representations, i.e. states capture all the necessary information, with no need of more detailed structures
- Solutions to reach a goal are searched among sequences of *state*→*action(s)*, often within graph representations
- Search algorithms can be divided in uninformed and informed:
  - **uninformed** algorithms do not know how “far” is the goal
  - **informed** algorithms have an estimate of the goal distance



# Problem formulation

A **problem** is defined by the following

- **set of possible states**
  - e.g.  $\{Arad, Timisoara, Zerind, Sibiu, \dots\}$
- **initial state**
  - e.g. “at Arad”
- (set of) **goal state**, can be
  - explicit, e.g.,  $x$  = “at Bucharest”
  - implicit, e.g.,  $NoDirt(x)$
- **set of actions**
  - e.g.  $Actions(Arad) = \{ToTimisoara, ToZerind, ToSibiu\}$
- **transition model**
  - e.g.  $Result(Arad, ToZerind) = Zerind$
- **action cost function** and (additive) **path cost**
  - $c(x, a, y)$  is the cost to execute action  $a$  from state  $x$  to reach state  $y$  (assumed  $\geq 0$ )
  - path cost could be sum of distances, number of actions executed, etc

A **solution** is a sequence of actions leading from the initial state to a goal state

# Example: vacuum world

- **States?**
  - *Dirt* and robot *Location* (ignoring dirt amounts)
  - How many? 2 robot locations x  $2^2$  dirt possibilities
  - Any initial state

- **Actions?**
  - *Left, Right, Suck, NoOp*

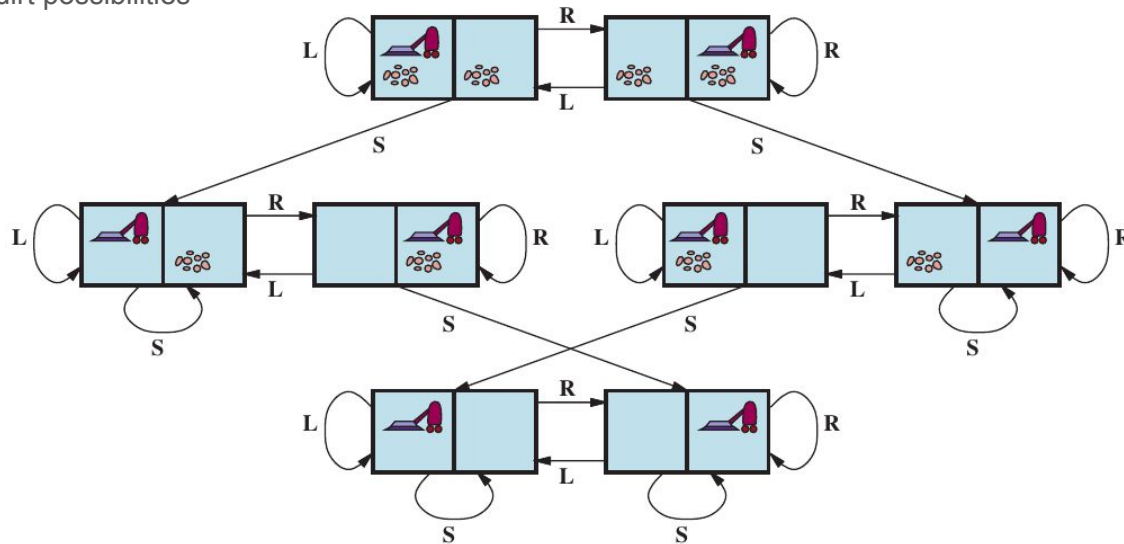
- **Transition model?** (see figure →)

- **Goal?**

- no dirt

- **Path cost?**

- 1 per action (0 for *NoOp*)

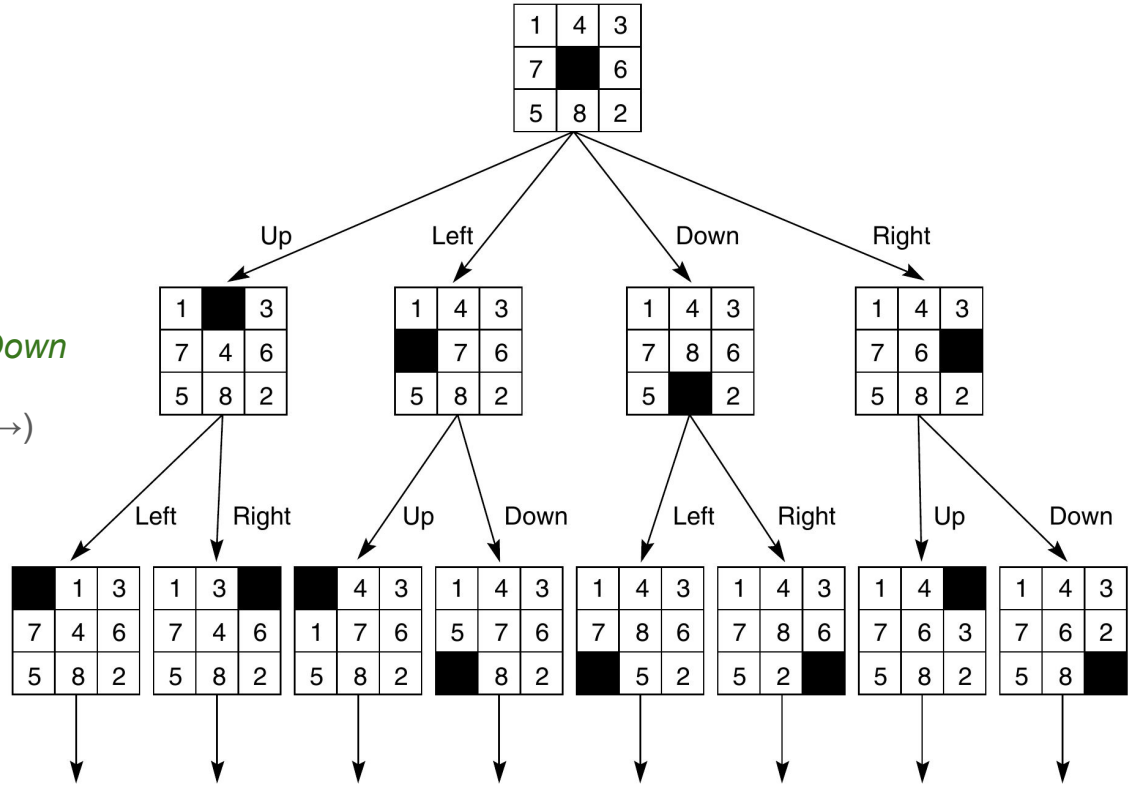




# Example: 8-puzzle

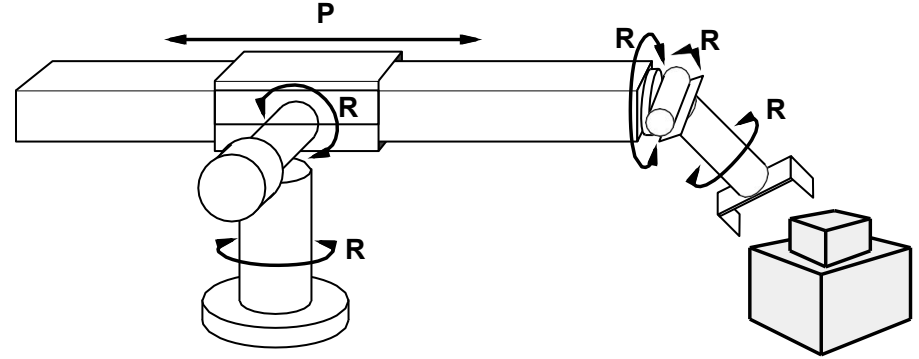
- **States?**
  - integer locations of tiles
- **Actions?**
  - move blank *Left, Right, Up, Down*
- **Transition model?** (see figure →)
- **Goal?**
  - e.g. 

|   |   |   |
|---|---|---|
|   | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |
- **Path cost?**
  - 1 per move



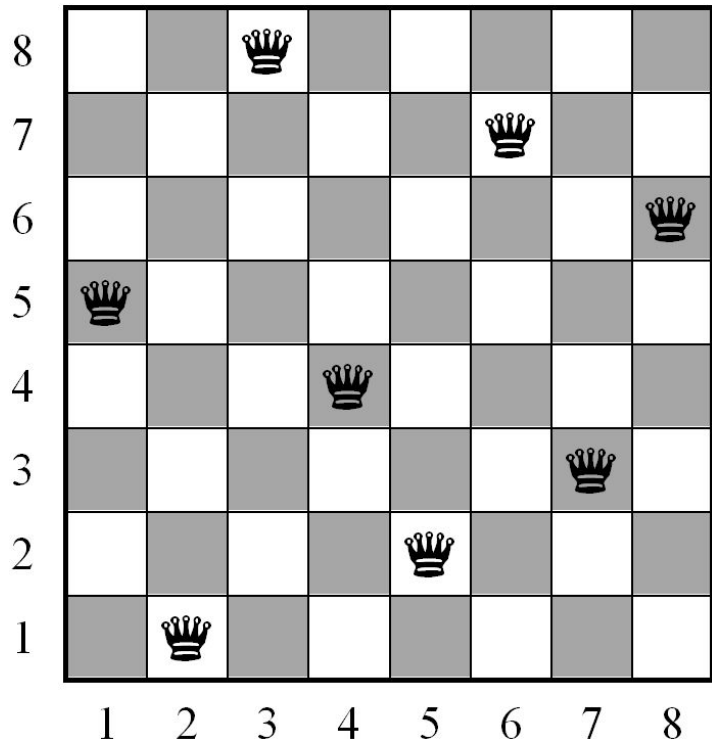
# Example: robotic assembly

- **States?**
  - **real-valued** coordinates of robot joint angles
  - parts of the object to be assembled
- **Actions?**
  - **continuous** motions of robot joints
- **Transition model?**
  - kinematic and dynamic models
- **Goal?**
  - complete assembly
- **Path cost?**
  - time to execute



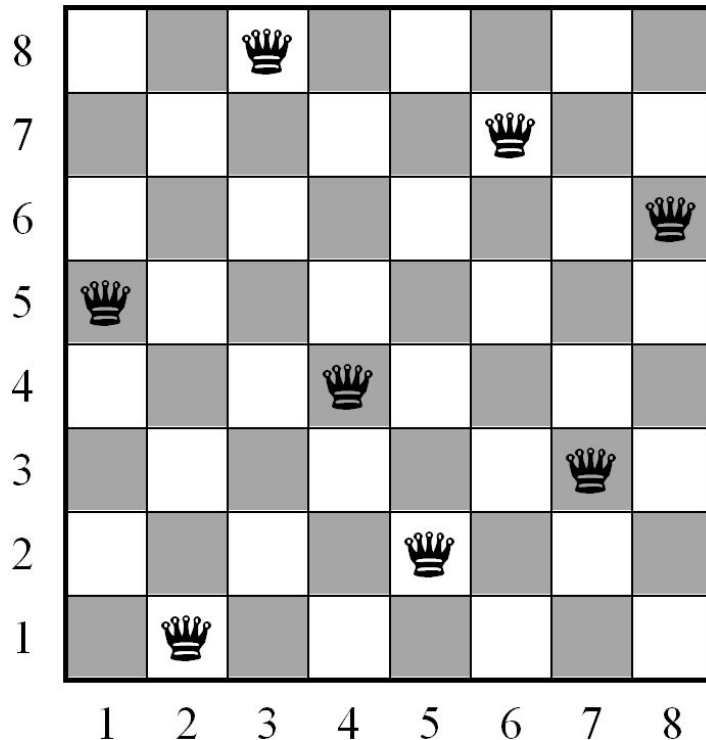
# Example: 8-queens problem

- **States?**
  - **any** arrangement of 0 to 8 queens on the board is a state
  - initial state: no queens on the board
- **Actions?**
  - add a queen to **any** empty square
- **Transition model?**
  - returns the board with a queen added to the specified square
- **Goal?**
  - Two goals: 8 queens are on the board AND none attacked
- **Path cost?**
  - N/A



# Example: 8-queens problem (better formulation)

- Previous formulation:  $64 \cdot 63 \cdot \dots \cdot 57 \approx 1.8 \times 10^{14}$  possible sequences to investigate!
- Better formulation: prohibit placing a queen in any square that is already attacked
- **States?**
  - all possible arrangements of  $n$  queens ( $0 \leq n \leq 8$ ), **one per column in the leftmost  $n$  columns, with no queen attacking another**
- **Actions?**
  - add a queen to any square **in the leftmost empty column such that it is not attacked by any other queen**
- This formulation reduces the 8-queens state space from  $1.8 \times 10^{14}$  to just **2,057**, and solutions are easy to find

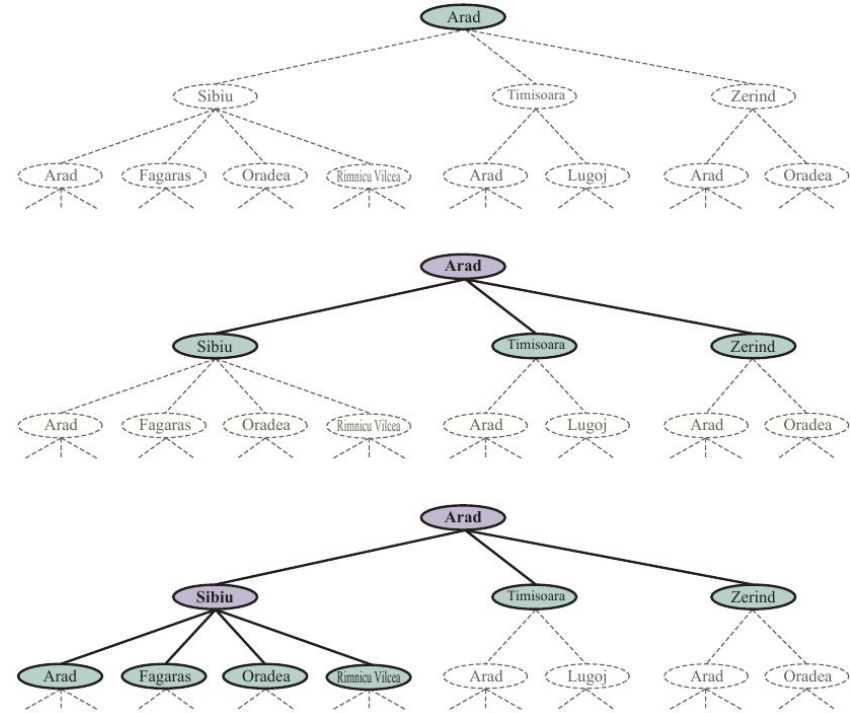


# Tree search algorithms

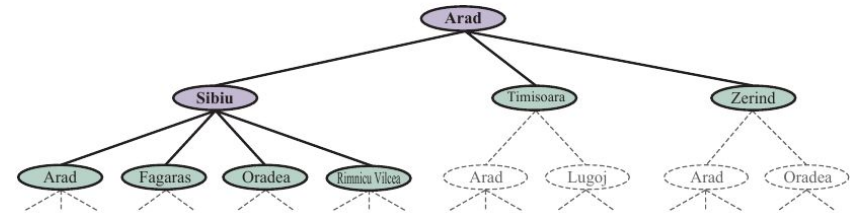
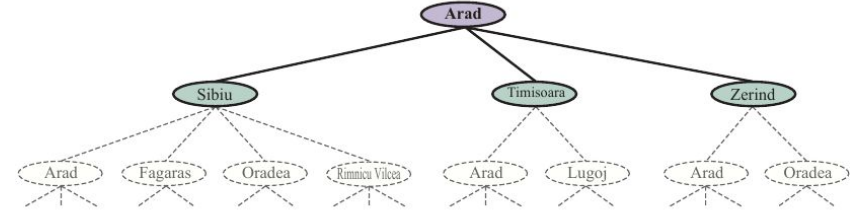
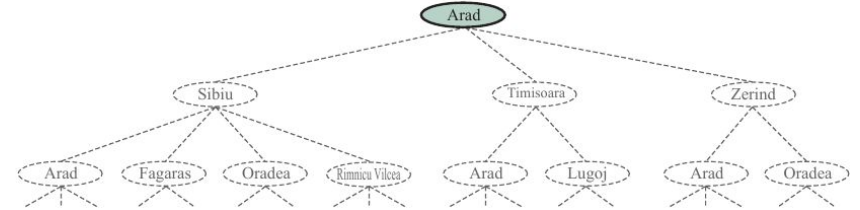
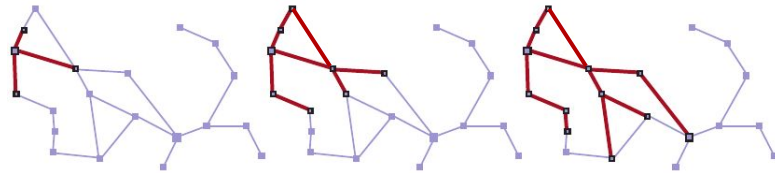
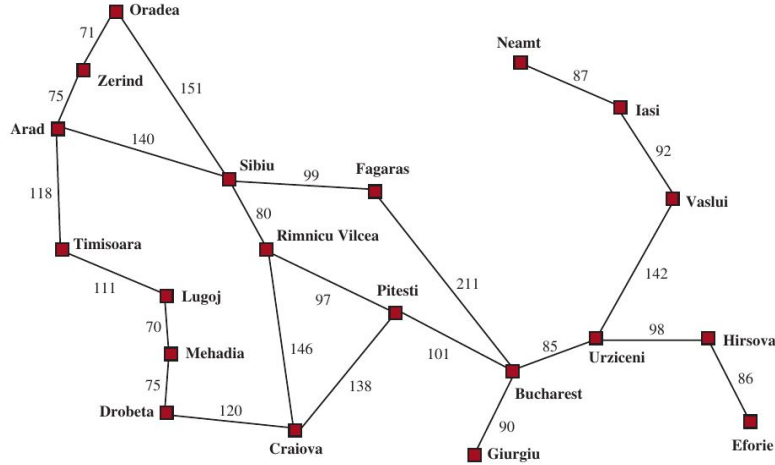
## Basic idea

- Offline, **simulated exploration** of state space by **generating successors of already-explored states** (a.k.a. expanding states)

```
function Tree-Search(problem, strategy) returns a solution, or failure
  initialize the search tree using the initial state of problem
  loop do
    if there are no candidates for expansion then return failure
    choose a leaf node for expansion according to strategy
    if the node contains a goal state then return the corresponding solution
    else expand the node and add the resulting nodes to the search tree
  end
```

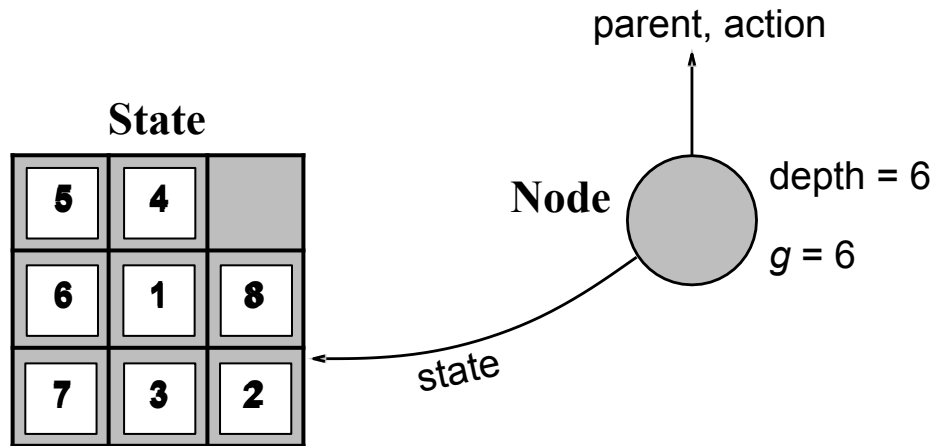


# Tree search example



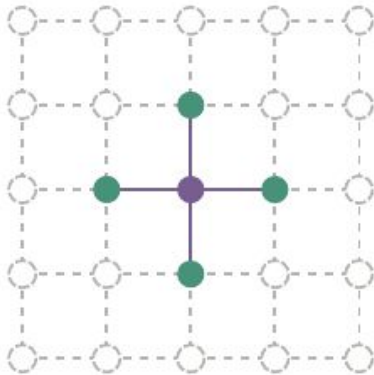
# States vs. nodes

- A **state** is (a representation of) a physical configuration
- A **node** is a data structure constituting part of a search tree
  - includes **state**, **parent**, **action**, **children** (if any), **depth  $d$** , **path cost  $g$**
- States do not have parents, children, depth, or path cost
- Two different nodes can contain the same state!

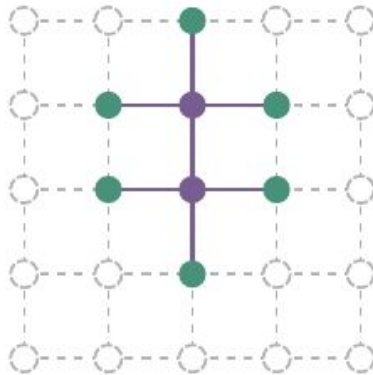


# Frontier of the search tree

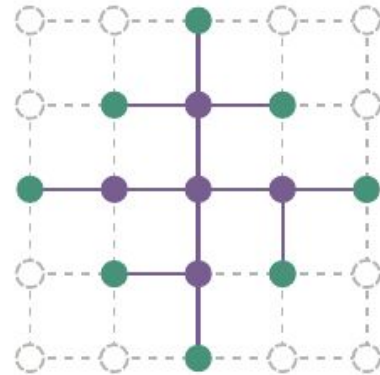
- The unexpanded (green) nodes constitute the **frontier** of the search tree
  - The frontier separates the **state-space graph** in two regions: visited and not
  - Solutions are checked when nodes selected for expansion, not when added to the frontier



(a)



(b)

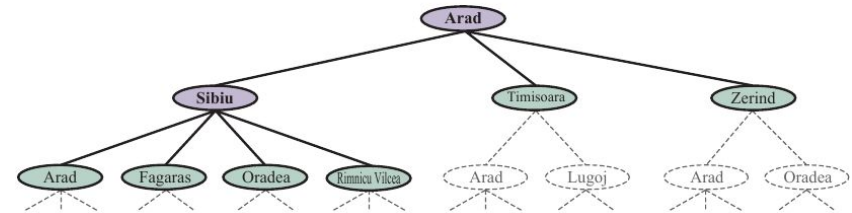
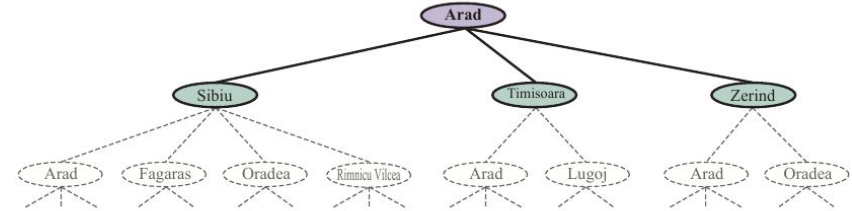
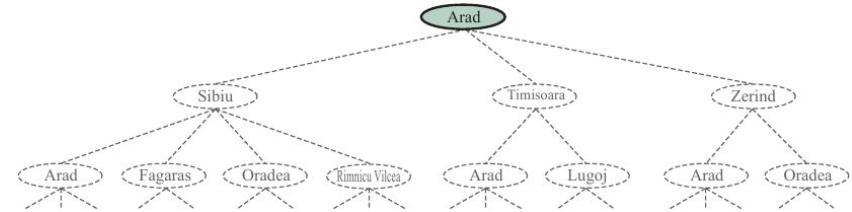


(c)



# Frontier example

- The initial frontier is  
 $\{\text{Arad}\}$
- At the next time step it's  
 $\{\text{Sibiu}, \text{Timisoara}, \text{Zerind}\}$
- Next it's  
 $\{\text{Arad}, \text{Fagaras}, \text{Oradea}, \text{Rimnicu Vilcea}, \text{Timisoara}, \text{Zerind}\}$



# Search strategies

- A **strategy** (search algorithm) is defined by picking the order of node expansion
- Strategies are evaluated along the following dimensions:
  - **completeness** — does it always find a solution if one exists?
  - **time complexity** — (proportional to) number of nodes generated/expanded
  - **space complexity** — max number of nodes in memory
  - **optimality** — does it always find a least-cost solution?
- Time and space complexity in terms of
  - $b$  = maximum branching factor of the search tree
  - $d$  = depth of least-cost solution
  - $m$  = maximum depth of state space (could be  $\infty$ )



# Uninformed search strategies

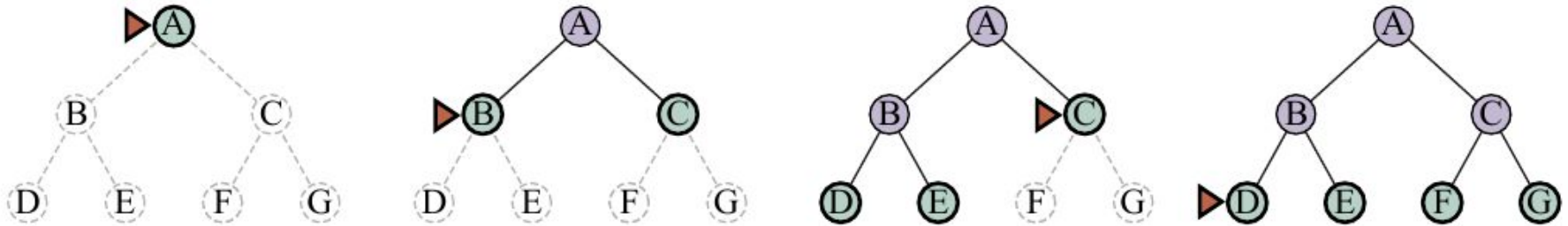
**Uninformed** strategies use only the information available in the problem definition

- **Breadth-first** search
- **Uniform-cost** search
- **Depth-first** search
- **Depth-limited** search
- **Iterative deepening** search



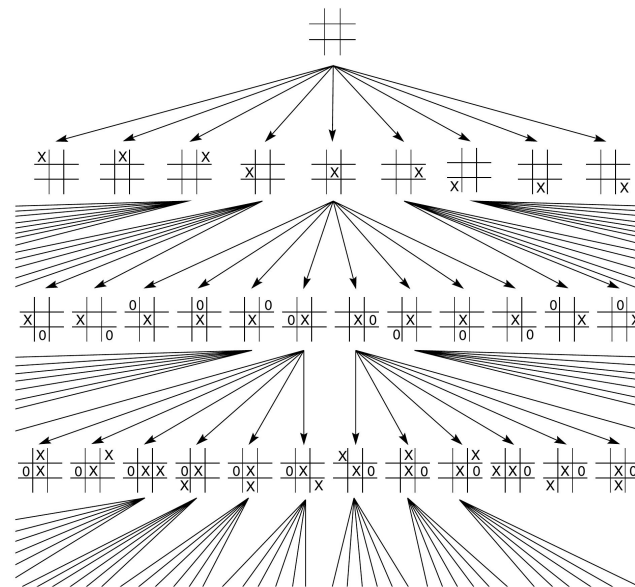
# Breadth-first search

- Expand shallowest unexpanded node
  - All actions have the **same cost**
  - The frontier is a **FIFO queue**, i.e. new successors go at end
  - In the figure:
    - Frontier = {**A**} → Frontier = {**B**, C} → Frontier = {**C**, D, E} → Frontier = {**D**, E, F, G}



# Properties of breadth-first search

- **Complete?**
  - **Yes** (if branching factor  $b$  is finite)
- **Time?**
  - $b + b^2 + b^3 + \dots + b^d = O(b^d)$
- **Space?**
  - $O(b^d)$  – keeps every node in memory
- **Optimal?**
  - **Yes**, if step cost = 1, but not in general



Space is the big problem!

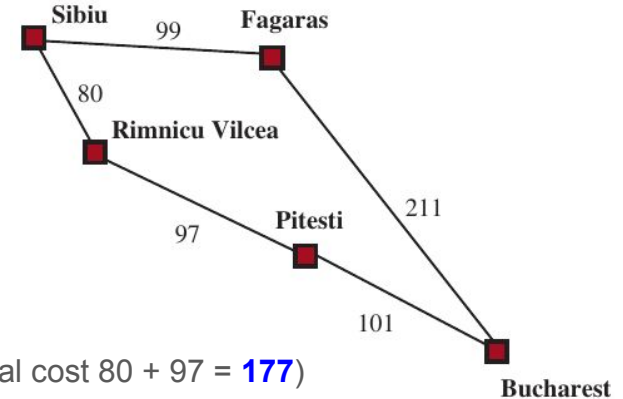
# Uniform-cost search (Dijkstra's algorithm)

- Expand least-cost node at the frontier

- The frontier is a queue ordered by path cost (lowest first)
- Equivalent to breadth-first if step costs all equal

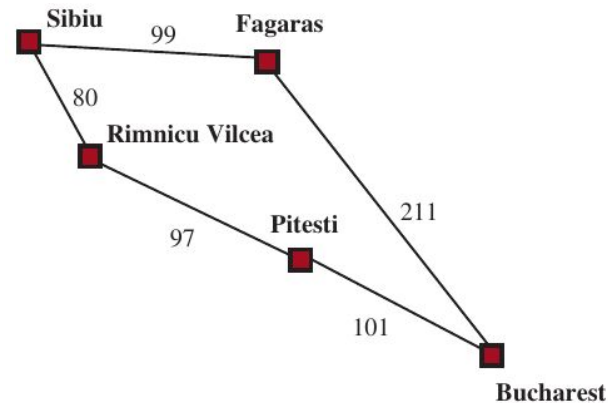
- Example – get from Sibiu to Bucharest:

1. successors of **Sibiu** are **Rimnicu Vilcea** (**80**) and **Fagaras** (**99**)
2. the least-cost, **Rimnicu Vilcea**, is expanded next, adding **Pitesti** (total cost  $80 + 97 =$  **177**)
3. the least-cost node is now **Fagaras**, so it is expanded, adding **Bucharest** (total cost  $99 + 211 =$  **310**) – **not finished yet** because goal tested only when *expanded*, not added the frontier
4.  $177 < 310$ , so **Pitesti** is expanded, creating a second path to **Bucharest** ( $177 + 101 =$  **278**)
5.  $278 < 310$ , so the path **Sibiu**→**Rimnicu Vilcea**→**Pitesti**→**Bucharest** replaces the one through **Fagaras**
6. **Bucharest** is then considered for expansion and found to be the goal



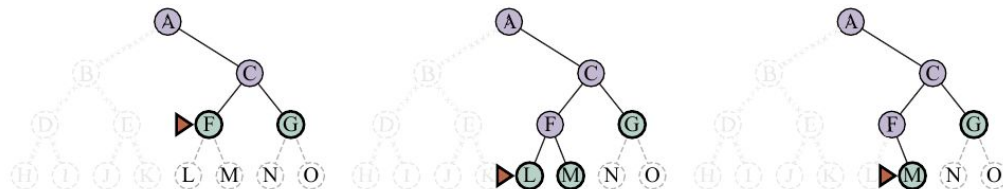
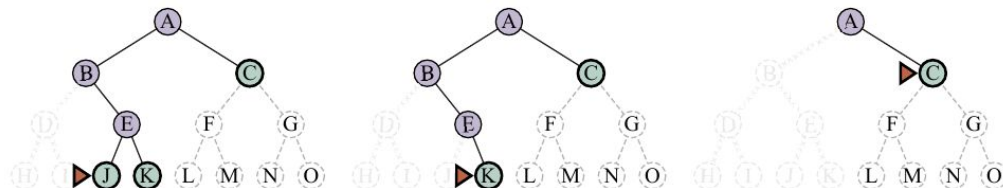
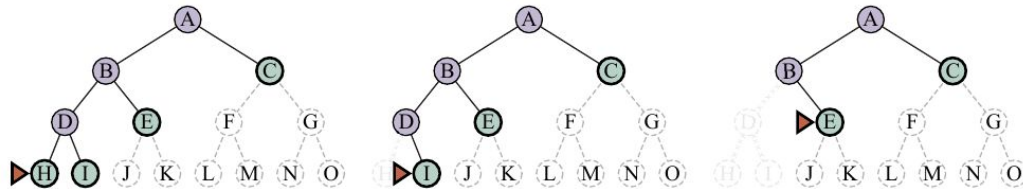
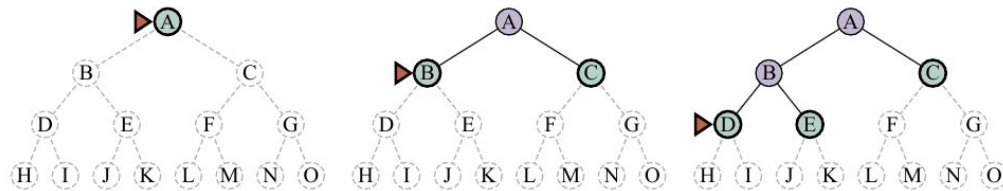
# Properties of uniform-cost search

- **Complete?**
  - **Yes**, if step cost  $\geq \epsilon$  (paths get always longer)
- **Time?**
  - can be much worse than  $O(b^d)$  for small step costs
- **Space?**
  - can be much worse than  $O(b^d)$  for small step costs
- **Optimal?**
  - **Yes** – nodes expanded in increasing order of path cost  $g$



# Depth-first search

- Expand deepest  
unexpanded node
  - The frontier is a LIFO queue, i.e. put successors at front
  - In the example:
    1. Frontier = {**A**}
    2. Frontier = {**B**, C}
    3. Frontier = {**D**, E, C}
    4. Frontier = {**H**, I, E, C}
    5. ...





# Properties of depth-first search

- **Complete?**

- **No**: fails in infinite-depth spaces and spaces with loops
  - can be modified to avoid repeated states along path, or...
  - set threshold to max depth → **depth-limited search**

- **Time?**

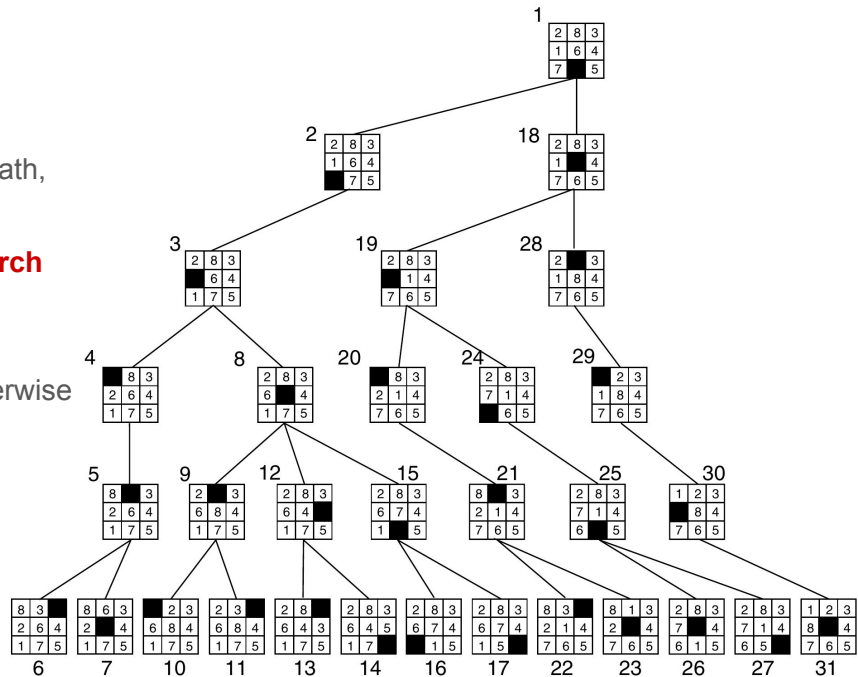
- $O(b^m)$ : terrible if max depth  $m$  is much larger than  $d$ , otherwise it could be much faster than breadth-first

- **Space?**

- $O(bm)$ , i.e. linear space,  $\ll O(b^d)$

- **Optimal?**

- **No**, stops at first solution found



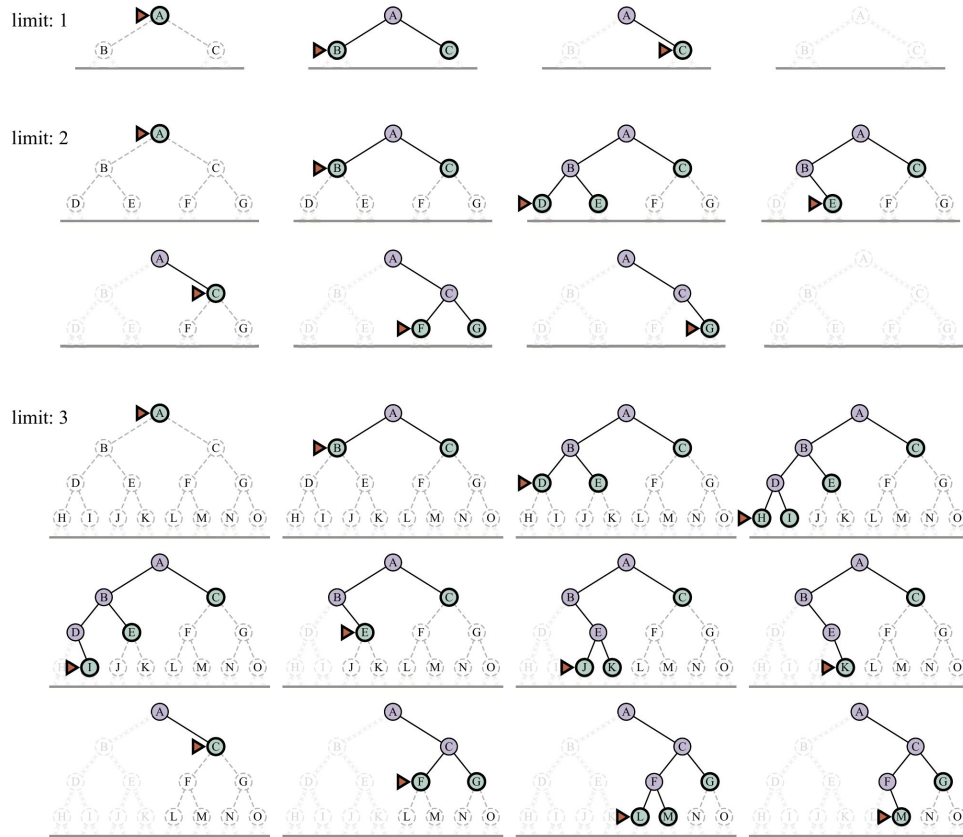
E.g. depth-limited search with  $d^* = 5$

Goal

# Iterative deepening search

- Similar to **depth-limited search**, but **iteratively increasing the limit**, e.g.

- execute depth-limited search with max depth = 1
- execute depth-limited search with max depth = 2
- execute depth-limited search with max depth = 3
- ...



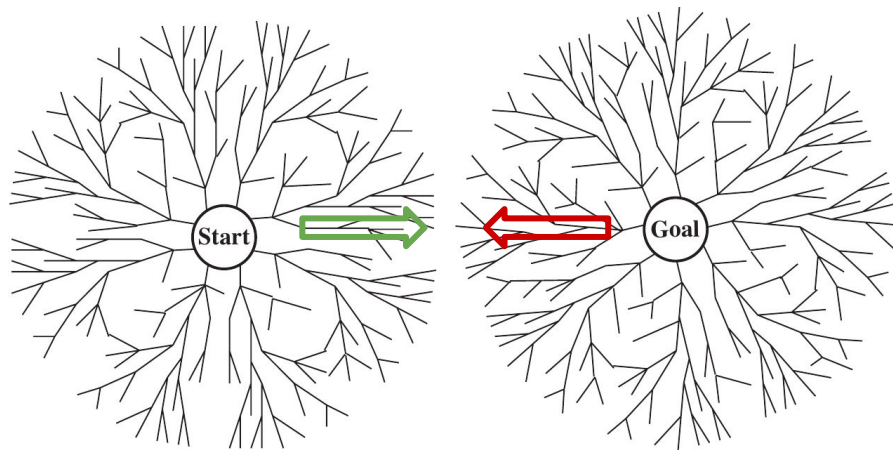
# Properties of iterative deepening search

- Complete?
  - Yes
- Time?
  - $db + (d - 1)b^2 + \dots + b^d = O(b^d)$  – asymptotically identical to breadth-first search (if there is a solution)
    - E.g. numerical comparison (# nodes) for  $d = 5$  and  $b = 10$ , when solution at far right leaf:  
Iterative Deepening Search:  $N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$   
Breadth First Search:  $N(\text{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110$
- Space?
  - $O(bd)$
- Optimal?
  - Could be, if action costs are identical

# Bidirectional search

- Idea – **two simultaneous searches**:
  - one **forward** from the start
  - one **backward** from the goal
- Implemented with **breadth-first** or **uniform-cost** search
- The search is successful if the two frontiers meet
- The motivation is that

$$O(b^{d/2}) + O(b^{d/2}) \ll O(b^d)$$



# Properties of bidirectional search

- **Complete?**
  - **Yes**, if  $b$  and state space are finite
- **Time?**
  - $O(b^{d/2})$
- **Space?**
  - $O(b^{d/2})$
- **Optimal?**
  - Could be, if action costs are identical
- Intuitive application to problems with single goal state (e.g. best route from Arad to Bucharest)
- If there are several explicit goal states (e.g. two dirt-free rooms in vacuum world), add a dummy goal state whose immediate predecessors are all the actual goal states
- If the goal is an abstract description (e.g. “no queen attacks another queen”), then bidirectional search is difficult to use

# Summary

- A problem consists of five parts: the **initial state**, a set of **actions**, a **transition model** describing the results of those actions, a set of **goal states**, and an **action cost function**
- **Uninformed search** methods have access only to the **problem definition**. Algorithms build a search tree in an attempt to find a solution

**TO BE  
CONTINUED....→**

# Conclusions

- Suggested reading  
Chapter 3, sec. 3.1-3.4 of Russell & Norvig
- Next lecture  
Solving problems by searching (2nd part)
- Questions?

